

Claude Folder Structure

FRONTMATTER

title: Claude Folder Structure
video no: 7 of 30
series: Building an AI Business From Scratch
voice_reference: ../../Input/4. Resources/2.
scripting_resources/voice_reference/nate_herk_voice_reference.md
video_type: tutorial
hook_type: Problem-Statement
script_versions: A + B
status: DRAFT

REFERENCE VIDEOS

- CLAUDE CODE FULL COURSE 4 HOURS: Build & Sell (2026) (<https://www.youtube.com/watch?v=QoQBzR1NIql>) -- Nick Saraev -- Watch the project structure section; notice how he separates instructions from outputs and why
- Claude Code Project Structure Best Practices (<https://uxplanet.org/claude-code-project-structure-best-practices-5a9c3c97f121>) -- Nick Babich (UX Planet) -- Read for the reasoning behind Input/Output separation and what breaks without it
- Anatomy of the .claude Folder (<https://codewithmukesh.com/blog/anatomy-of-the-claude-folder/>) -- Code With Mukesh -- Watch for the breakdown of every auto-generated Claude file and what each one controls

THE ONE THING

Claude Code works best when your folder structure tells it exactly what belongs where -- a clear structure is a better tool than a better prompt.

HOOK

Most Claude Code frustration comes from one source: Claude does not know where things belong in your project. Here is the folder structure that fixes that permanently.

TARGET VIEWER

Someone who has Claude Code installed and working but finds it placing files inconsistently, asking where things should go, or struggling to navigate the project between sessions.

PAYOFF

After watching, the viewer has a clear, practical folder structure they can apply to any Claude Code project immediately -- one that makes Claude smarter about their project from the very first session.

BEAT STRUCTURE

Beat -- Why Structure Matters More in AI Projects

Point: Claude Code reads your folder structure to understand your project. A messy structure gives Claude bad context; a clear structure gives it a map.

Visual: Two project folders side by side: messy (files dumped in root) vs organised (clear folders) -- show Claude's confusion in one vs clear navigation in the other

Target words: ~120w

Beat -- The Core Principle -- Input vs Output

Point: The most important structural decision is separating what Claude reads (inputs, instructions) from what Claude produces (outputs). Mixing them creates ambiguity about what can be changed.

Visual: Diagram: Input/ and Output/ as the two root folders -- arrow showing Claude reads from Input, writes to Output

Target words: ~160w

Beat -- The Standard Pattern

Point: For any Claude Code project: /Input holds instructions, agents, skills, rules, resources. /Output holds everything Claude produces. /Quick Access holds reference files needed fast.

Visual: Folder tree diagram expanding each section with example files under each node

Target words: ~190w

Beat -- CLAUDE.md -- The File That Ties It Together

Point: CLAUDE.md in the project root tells Claude about the structure, the conventions, and the rules. Without it, Claude guesses. With it, Claude navigates like it already knows the project.

Visual: CLAUDE.md file open in editor showing a simple project structure section -- split with terminal showing Claude referencing it correctly

Target words: ~150w

Beat -- Applying It to Real Projects

Point: Same principle, three contexts: a content creation pipeline, an automation project, a personal tool. The Input/Output pattern scales to any project type.

Visual: Three minimal folder trees side by side showing the pattern adapted to each context

Target words: ~140w

Beat -- CTA Bridge

Point: Structure is set. Next: Claude Cowork -- the new Claude product for collaborative AI work that most people have not found yet.

Visual: Preview clip from Claude Cowork Explained

Target words: ~60w

ANALOGY BANK

- Folder structure for Claude: Like a filing system for a human assistant -- if your assistant doesn't know where contracts go vs drafts vs templates, they put everything in one pile and spend half their time asking where things belong.
- Input/Output separation: Like a kitchen where prep and plating are physically separate -- mixing them creates chaos; keeping them apart lets each stage run at full speed without interfering with the other.

THESIS LINE LOCATION

Beat 2 -- The insight is: your folder structure is a form of communication -- a clear project tells Claude what matters, where things go, and what can be changed vs what must be preserved.

FRAMEWORK PHRASE

Input, output, clear

SPECIFICITY BANK

- CLAUDE.md goes in the project root directory
- /Input holds: agents, skills, rules, resources, tools
- /Output holds: produced content and artifacts
- .claude/ folder (auto-created by Claude Code) stores settings, permissions, and local config
- Quick Access subfolder for frequently referenced files Claude needs fast

CONSTRAINTS

- Do not go into CLAUDE.md content in depth -- that is the next-next video
- Do not explain permissions -- that is a later video
- Keep it practical -- show the structure, not the philosophy behind it
- Do not cover the .claude/ auto-generated folder in depth -- mention it exists, move on

CTA DIRECTION

Structure is sorted. Next: Claude Cowork -- the new way to use Claude for real work that most people have not found yet.

PERSONAL ANGLE [DANIEL TO FILL]

Suggested: The folder structure you settled on after trying several approaches -- what you changed, why, and what stopped breaking once you did.

VULNERABILITY MOMENT [DANIEL TO FILL]

Suggested: A time when Claude placed files in completely the wrong place because your structure was ambiguous -- and how adding clear Input/Output separation fixed it immediately.

Note: belongs in Beat 1